

Semana 15 - P1

Repaso para la Solemne 3

Análisis de datos, algoritmos y herramientas numéricas

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 15, Sesión 1 (Sesión 29)

Preparación para la Solemne 3

Repaso de Pandas

Repaso de Algoritmos

Repaso de `roots` y `polyfit`

Cierre

Preparación para la Solemne 3

Al finalizar la sesión, cada estudiante debería poder:

- reconocer el tipo de problema presentado;
- seleccionar una herramienta apropiada;
- dividir una solución en pasos verificables;
- leer y explicar código corto;
- interpretar tablas, gráficos y resultados numéricos;
- detectar errores frecuentes antes de entregar.

Preparación para la Solemne 3

Mapa del repaso

Pandas

Trabajar con datos organizados en tablas.

Algoritmos

Comprender pasos, costo y escalamiento.

NumPy

Resolver polinomios y ajustar mediciones.

Pregunta central

¿Qué información tengo y qué resultado necesito obtener?

Preparación para la Solemne 3

Estrategia general de resolución

1. **Leer:** identificar datos, variables y resultado solicitado.
2. **Clasificar:** reconocer el tipo de problema.
3. **Planificar:** escribir los pasos antes del código.
4. **Programar:** resolver una etapa a la vez.
5. **Verificar:** revisar tamaños, valores y unidades.
6. **Interpretar:** responder con palabras la pregunta original.

Buena práctica

Una celda corta que funciona es mejor punto de partida que un programa largo sin verificar.

Repaso de Pandas

Idea

Un DataFrame es una tabla bidimensional con filas, columnas, nombres y tipos de datos.

- Cada columna representa una variable.
- Cada fila representa una observación.
- Las columnas pueden contener números, texto o valores lógicos.
- Los valores faltantes suelen representarse con **NaN**.

Antes de calcular

Debemos conocer la estructura y la calidad de los datos.

Repaso de Pandas

Inspección inicial

```
1 print(df.head())
2 print(df.shape)
3 print(df.columns)
4 print(df.dtypes)
5 print(df.info())
```

- `head()`: muestra las primeras filas.
- `shape`: entrega (`filas, columnas`).
- `columns`: muestra los nombres disponibles.
- `dtypes`: informa el tipo de cada columna.

Repaso de Pandas

Seleccionar columnas

Una columna se selecciona con un nombre; varias columnas, con una lista de nombres.

```
1 temperatura = df["temperatura"]
2
3 subtabla = df[
4     ["nombre", "temperatura", "distancia"]
5 ]
```

Error frecuente

Para varias columnas se necesitan dos pares de corchetes: uno para seleccionar y otro para construir la lista.

Repaso de Pandas

Filtrar filas

```
1 cercanas = df[df["distancia"] < 20]
2
3 seleccion = df[
4     (df["temperatura"] > 6000) &
5     (df["distancia"] < 20)
6 ]
```

- Cada comparación produce valores **True** o **False**.
- Pandas conserva las filas donde la condición es verdadera.
- En filtros múltiples usamos **&** y paréntesis.

Repaso de Pandas

Datos faltantes

```
1 print(df.isna().sum())
2
3 df_limpio = df.dropna(
4     subset=["temperatura"]
5 ).copy()
6
7 df["temperatura"] = df["temperatura"].fillna(
8     df["temperatura"].median()
9 )
```

Decisión

Eliminar y rellenar no son operaciones equivalentes. La elección depende del significado de los datos.

Tendencia central

- Media
- Mediana

La mediana es menos sensible a valores extremos.

Dispersión

- Desviación estándar
- Mínimo y máximo
- Cuantiles

La dispersión describe cuánto varían los datos.

Repaso de Pandas

Agrupar y resumir

```
1 resumen = df.groupby("clase").agg(  
2     temperatura_media=("temperatura", "mean"),  
3     magnitud_media=("magnitud", "mean"),  
4     cantidad=("nombre", "count")  
5 )  
6  
7 print(resumen)
```

Interpretación

groupby separa las filas por categoría y **agg** calcula estadísticas dentro de cada grupo.

Repaso de Pandas

Elegir un gráfico

- **Histograma:** observar la distribución de una variable.
- **Scatter:** estudiar la relación entre dos variables numéricas.
- **Boxplot:** comparar distribuciones entre categorías.
- **Barras:** comparar cantidades o resúmenes por grupo.

Un gráfico debe incluir

Etiquetas, unidades, título o contexto y, cuando corresponda, una leyenda.

1. Cargar o crear el DataFrame.
2. Inspeccionar filas, columnas y tipos.
3. Detectar y tratar valores faltantes.
4. Seleccionar las variables relevantes.
5. Filtrar las observaciones necesarias.
6. Agrupar y calcular estadísticas.
7. Visualizar.
8. Escribir una conclusión.

Resultado esperado

No basta con imprimir una tabla: debemos explicar qué información entrega.

Repaso de Algoritmos

Repaso de Algoritmos

¿Qué es un algoritmo?

Definición práctica

Un algoritmo es una secuencia finita y ordenada de instrucciones para resolver un problema.

Un algoritmo debe:

- recibir datos de entrada;
- realizar operaciones claramente definidas;
- terminar;
- producir un resultado;
- funcionar para más de un caso particular.

Lineal

- Revisa uno por uno.
- Funciona en listas desordenadas.
- Costo aproximado: $\mathcal{O}(n)$.

Binaria

- Descarta la mitad en cada paso.
- Requiere una lista ordenada.
- Costo aproximado: $\mathcal{O}(\log n)$.

Considere la lista:

[2, 5, 8, 12, 20, 31, 45].

Para buscar 20:

1. comparar con el elemento central;
2. decidir izquierda o derecha;
3. repetir sobre la mitad restante.

Importante

Si la lista no está ordenada, la decisión de descartar una mitad deja de ser válida.

Repaso de Algoritmos

Estructura de la recursividad

Una función recursiva contiene:

Caso base

Situación sencilla que puede resolverse directamente y detiene las llamadas.

Caso recursivo

La función se llama a sí misma con una versión más pequeña del problema.

Riesgo

Sin un caso base alcanzable, Python termina con **RecursionError**.

```
1 def suma_recursiva(lista):  
2     if len(lista) == 0:  
3         return 0  
4     return lista[0] + suma_recursiva(lista[1:])
```

Para [4, 7, 2]:

$$S([4, 7, 2]) = 4 + S([7, 2]) = 4 + 7 + S([2]) = 4 + 7 + 2 + S([]) = 13.$$

Repaso de Algoritmos

Bubble Sort

Idea

Comparar elementos vecinos e intercambiarlos cuando están en el orden incorrecto.

- En cada pasada, un elemento grande avanza hacia el final.
- Utiliza dos bucles anidados.
- Es fácil de comprender e implementar.
- Para listas grandes realiza muchas comparaciones.

Costo típico: $\mathcal{O}(n^2)$.

Divide y conquista

1. Dividir la lista en dos mitades.
 2. Ordenar recursivamente cada mitad.
 3. Mezclar las dos mitades ordenadas.
- El caso base es una lista con cero o un elemento.
 - La función **merge** combina listas ya ordenadas.
 - Su costo típico es $\mathcal{O}(n \log n)$.

Repaso de Algoritmos

Complejidad y escalamiento

Algoritmo	Complejidad	Al duplicar n
Búsqueda lineal	$\mathcal{O}(n)$	trabajo aproximado $\times 2$
Búsqueda binaria	$\mathcal{O}(\log n)$	aumenta muy poco
Bubble Sort	$\mathcal{O}(n^2)$	trabajo aproximado $\times 4$
Merge Sort	$\mathcal{O}(n \log n)$	algo más que $\times 2$

Interpretación

La complejidad describe una tendencia de crecimiento, no un tiempo exacto en segundos.

Podemos estudiar un algoritmo de dos formas:

Tiempo

Medir cuánto tarda con
`time.time()`.

Depende del computador y de
otros procesos.

Operaciones

Contar comparaciones,
intercambios o llamadas.

Describe directamente el
trabajo realizado.

Al probar varios tamaños podemos graficar costo versus n .

Repaso de roots y polyfit

Repaso de `roots` y `polyfit`

Resolver ecuaciones con **`np.roots`**

`np.roots` encuentra los valores donde un polinomio es cero.

$$p(x) = a_n x^n + \dots + a_2 x^2 + a_1 x + a_0.$$

La entrada es:

$$[a_n, \dots, a_2, a_1, a_0].$$

Orden obligatorio

Los coeficientes se escriben desde la potencia mayor hasta el término independiente.

Repaso de roots y polyfit

Raíces reales y soluciones físicas

```
1 coef = [-4.9, 16, 2]
2 raices = np.roots(coef)
3
4 for raiz in raices:
5     if abs(raiz.imag) < 1e-10:
6         tiempo = raiz.real
7         if tiempo >= 0:
8             print(tiempo)
```

- Una raíz compleja puede ser matemáticamente correcta.
- Una raíz real negativa puede no tener sentido físico.
- El contexto determina cuáles soluciones se aceptan.

Repaso de roots y polyfit

Ajustar datos con ***np.polyfit***

Si tenemos mediciones (x_i, y_i) , buscamos un polinomio que represente su tendencia.

Ajuste lineal

$$y = mx + b, \quad m, b = \text{np.polyfit}(x, y, \text{deg}=1).$$

Ajuste cuadrático

$$y = ax^2 + bx + c, \quad a, b, c = \text{np.polyfit}(x, y, \text{deg}=2).$$

Repaso de roots y polyfit

Residuos y coeficiente R^2

```
1 y_fit = m*x + b
2 residuos = y - y_fit
3
4 SS_res = np.sum(residuos**2)
5 SS_tot = np.sum((y - np.mean(y))**2)
6 R2 = 1 - SS_res/SS_tot
```

- El residuo es la diferencia entre dato y modelo.
- R^2 cercano a 1 indica que el modelo explica gran parte de la variación.
- Un valor alto no reemplaza la inspección del gráfico.

Repaso de `roots` y `polyfit`

Combinar **`polyfit`** y **`roots`**

Podemos ajustar datos y luego resolver una pregunta sobre el modelo.

Ejemplo de flujo

1. Ajustar $y = ax^2 + bx + c$.
2. Preguntar cuándo el modelo alcanza $y = 20$.
3. Reescribir:

$$ax^2 + bx + (c - 20) = 0.$$

4. Aplicar `np.roots([a, b, c - 20])`.
5. Filtrar soluciones según el contexto.

Cierre

- **Pandas:** inspeccionar, limpiar, filtrar, agrupar e interpretar.
- **Algoritmos:** identificar caso base, ciclos, contadores y escalamiento.
- **np.roots:** ordenar coeficientes y filtrar raíces.
- **np.polyfit:** construir predicciones, residuos y gráfico.
- **Siempre:** incluir etiquetas, unidades y una conclusión.

Preparación para P2

La segunda parte aplicará este mapa conceptual en ejercicios tipo Solemne 3.